MPTA-Repair: Clock-Bound Repair for Multi-Property Timed Automata in Industrial Practice

Guangtong Zhou, Run Chang, Ji Wu*
Beihang University, Beijing, China
Email: zhougt@buaa.edu.cn, handsomeruncr@gmail.com, wuji@buaa.edu.cn

***Abstract*—Timed automata (TAs) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple predefined properties simultaneously. When verification fails, repairing the model is challenging because it requires modifying erroneous clock constraints to satisfy all properties while preserving functional equivalence. To address this, we present MPTA-Repair, an automated repair framework for multi-property TAs. The framework employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. To optimize search efficiency, it systematically exploits relationships among properties, counterexamples, and repair variables. Each candidate model is then validated through full-property regression verification and acceptability checks. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks derived from UPPAAL and the XtaBenchmarkSuite, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—Timed automata, automated model repair, MaxSMT, multi-property verification**

I. Introduction

Timed automata (TAs) are widely used to model and verify real-time systems whose correctness depends on both discrete behavior and quantitative timing constraints [1], [2]. They appear in railway control, communication protocols, medical cyber-physical systems, and aerospace scheduling [3]–[6]. In these systems, an alarm, message, pulse, actuation, or resource release may be functionally present but still incorrect if it occurs too early, too late, or for an invalid duration.

In most practical settings, timed-automata models are validated against sets of properties rather than isolated assertions. These properties jointly capture safety, bounded response, timeout handling, deadline satisfaction, alarm duration, and deadlock freedom [7]. When a property is violated, model checkers such as UPPAAL, Kronos, or opaal can produce a timed diagnostic trace (TDT), i.e., a timed counterexample that reaches a violating state [8]–[10]. A TDT is useful failure evidence, but it does not identify which numerical bound is wrong, whether the fault lies in a guard or an invariant, or how the bound should be changed.

The repair problem becomes harder in the multi-property setting. A bound change that blocks one diagnostic trace may leave another violation feasible, break a property that was previously satisfied, or break functional equivalence with the original model. The running example in Section II illustrates this effect: repairing an early gear-engagement violation can make an acknowledgement deadline fail. A practical repair method must therefore generate candidate edits from local counterexamples, but accept them only after complete property regression and behavior-preservation checks.

Existing repair techniques provide an important foundation. Clock-bound repair encodes a TDT as linear real arithmetic, introduces variation variables for numerical clock bounds, and uses MaxSMT solving to compute small changes to guards and invariants [11]. TARTAR extends this idea with syntactic repair generation and an admissibility check based on untimed functional equivalence [11]. Related work also studies parametric timed automata, clock-guard repair, robustness analysis, and SMT-based repair [12]–[15]. However, TDT-based workflows are still commonly driven by one violated property or one diagnostic trace at a time [11].

This paper presents MPTA-Repair, an automated framework for multi-property clock-bound repair of timed automata. Given a faulty model, a property set, and diagnostic traces, MPTA-Repair searches for small changes to numerical bounds in transition guards and location invariants. The framework maintains a pool of property-trace constraints, uses relations among properties, counterexamples, and repairable bounds to guide MaxSMT-based candidate generation, and validates each candidate by full-property regression and a TARTAR-style admissibility check.

We evaluate MPTA-Repair on faulty models generated by clock-bound mutation. The evaluation includes benchmark models derived from UPPAAL and the XtaBenchmarkSuite, together with control-domain case-study models reconstructed from practical timed-automata examples. Compared with an iterative single-property baseline under the same editable clock-bound sites and global acceptance gates, MPTA-Repair improves repair success from 72% to 93% on the benchmark dataset and from 20% to 76% on the control-domain dataset.

Contributions. This paper makes four contributions: it formulates clock-bound repair as a multi-property and multi-counterexample problem; presents an iterative MaxSMT-based repair workflow guided by property, trace, and repair-variable relations; implements a prototype combining trace analysis, candidate generation, full-property regression, and admissibility checking; and evaluates the approach against an iterative single-property baseline while reporting practical lessons on regression validation, admissibility, and repair-space limitations.

The remainder of the paper is organized as follows. Section II defines the problem setting. Section III presents MPTA-Repair. Section IV reports the implementation and evaluation. Section V concludes the paper.

II. Preliminaries

A. Timed Automata and Repair Scope

Definition 1. A timed automaton is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I) \tag{1}$$

where L is a finite set of locations, ℓ_0 is the initial location, C is a finite set of clocks, Σ is a set of action labels, Θ is a finite set of transitions, and I maps each location to a clock invariant. A clock constraint is a conjunction of atoms $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and $b \in \mathbb{R}_{\geq 0}$. A transition $\theta = (\ell, g, a, R, \ell')$ moves from ℓ to ℓ' when guard g is satisfied, emits action a, and resets clocks in $R \subseteq C$. A network timed automaton is a parallel composition $\mathcal{N} = T_1 \parallel \cdots \parallel T_k$ with the synchronization semantics of the target modeling language, such as UPPAAL [8]. Delay steps must respect location invariants, whereas discrete steps are enabled by guards.

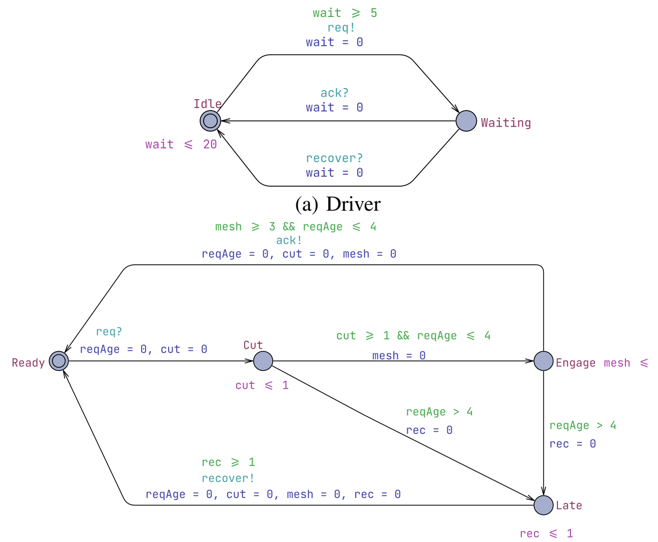
MPTA-Repair treats the network structure as fixed. The only repairable sites are numerical bound occurrences in transition guards and location invariants. A site s containing $x \sim b_s$ may be changed to $x \sim (b_s + \delta_s)$, where δ_s is a variation variable. The operator, clock reference, transition, location, synchronization label, reset set, data update, and property formula are not changed. This scope targets timing-parameter faults such as incorrect deadlines, timeouts, sampling periods, alarm durations, or minimum residence times; faults requiring reset, synchronization, data, or structural edits are outside the current repair space.

B. Diagnostic Traces and Multi-Property Motivation

Timed-automata verification in this paper is reduced to safety-style obligations. Direct examples include queries of the form $\mathrm{A}[]\,\psi$ and $\mathrm{A}[]\,\mathit{not\ deadlock}$. Bounded-response or deadline requirements are handled when monitor or observer templates encode them as reachable error locations [7].

Definition 2. A symbolic timed trace (STT) of a model is a finite transition sequence $S = \theta_0, \ldots, \theta_{n-1}$. A realization of S is a sequence of non-negative delays $d_0, \ldots, d_n$ and states $s_0, \ldots, s_{n+1}$ such that, for every $i < n$, the model can delay from s_i by d_i and then take θ_i to s_{i+1} while respecting guards, resets, invariants, and synchronizations; the final delay d_n leads from s_n to s_{n+1}. The STT is feasible if it has at least one realization. Given a safety property φ, a feasible STT is a timed diagnostic trace (TDT) for φ if at least one realization reaches a state that violates φ.

Example 1. Figure 1 shows the running gear-shift example. The Driver issues a request through $req!$, and the controller responds through $ack!$ or $recover!$. In the faulty controller,



(a) Driver

(b) GearController

Fig. 1. Simplified gear-shift example.

$cut \leq 1$ and $cut \geq 1$ allow engagement after one time unit, violating property φ_1, which requires at least two time units of torque cut. A single-property repair can change these bounds to 2. However, if the $mesh \leq 3$ and $mesh \geq 3$ bounds remain unchanged, the earliest acknowledgement occurs after $2 + 3 = 5$ time units, violating property φ_2, which requires acknowledgement within four time units. A multi-property repair must therefore adjust both cut and $mesh$ while preserving the same untimed request–engage–acknowledge behavior.

A TDT is failure evidence, not a root-cause explanation. It does not determine which numerical bound is faulty or whether the responsible occurrence is a guard or an invariant. TDT-based clock-bound repair encodes trace feasibility and property violation as linear arithmetic constraints over delays, valuations, and bound variations [11]. MaxSMT solving then prefers small repairs, typically by minimizing the number and magnitude of nonzero variations [16]. In the single-trace setting, the resulting candidate blocks the considered violating realization, but it does not prove that the whole model is repaired.

Definition 3. Let M_{bad} be a faulty timed-automata network and let $\Phi = \{\varphi_1, \ldots, \varphi_m\}$ be the complete property set used for validation. A clock-bound repair constructs M_{rep} by changing a small set of repairable bound occurrences. The repair is accepted only if the repaired bounds are well formed,

$$\begin{aligned} &M_{rep} \models \Phi \quad \text{and} \\ &\mathrm{Untimed}_{\Sigma_f}(M_{bad}) = \mathrm{Untimed}_{\Sigma_f}(M_{rep}) \end{aligned} \tag{2}$$

Here Σ_f is the functional alphabet used for admissibility after excluding monitor-only, observer-only, and internal labels when appropriate. This TARTAR-style criterion prevents repairs that satisfy properties merely by disabling required actions [11].

The objective in Definition 3 explains why MPTA-Repair accumulates property-trace obligations across refinement rounds. A bound edit that fixes one TDT may leave another violation feasible, break a previously satisfied property, or remove intended functionality. MPTA-Repair therefore uses TDT encodings for candidate generation only, and accepts a candidate only after full-property regression and functional-admissibility checking.

III. Approach

A. Overview

MPTA-Repair takes a timed-automata model M and a property set $\Phi = \{\varphi_1, \dots, \varphi_m\}$ as input. Instead of requiring manual variable specification, the framework extracts repairable bounds from numerical clock constraints in transition guards and location invariants. The final output is either a repaired model M' accompanied by concrete clock-bound edits, or a failure report if no valid full-property repair can be synthesized within the resource budget.

The framework operates on a local-generation and global-validation principle. While individual timed diagnostic traces (TDTs) drive candidate generation, validation remains global to ensure that fixing one violation does not introduce regressions elsewhere. To leverage the interdependence where multiple violations share identical timing structures, MPTA-Repair maintains a joint pool of property-trace obligations. This joint structure enables the framework to exploit relations among properties, counterexamples, and repairable bounds to optimize the search space and guide the MaxSMT solver toward minimal parameter assignments.

Operationally, each refinement round begins by verifying M against Φ to collect TDTs for any violations. If violations are detected, the framework identifies active repairable bounds and constructs a joint MaxSMT problem from the active obligation pool. The synthesized parameter assignment is instantiated into a candidate model, which undergoes global validation via full-property regression and an untimed functional-equivalence admissibility check. A rejected candidate contributes newly exposed diagnostic traces back into the pool to further constrain subsequent rounds, whereas a candidate passing both checks is returned as the valid repair M'. Figure 2 summarizes this counterexample-guided refinement loop from verification and relation-guided MaxSMT solving to global validation.

B. Joint Trace Encoding and Optimization Objective

MPTA-Repair follows the clock-bound variation strategy used in TARTAR [11]. Each modifiable numerical bound b_s in a transition guard or location invariant is associated with a variation variable δ_s. This ensures that the repaired constraint retains the original clock reference and comparison operator while incorporating a shifted value:

$$b'_s = b_s + \delta_s \tag{3}$$

The repair space is therefore strictly restricted to clock-bound constants, keeping transition structures and discrete updates unaltered.

MPTA-Repair builds on the trace-encoding formulation of TARTAR and extends it to the multi-property setting [11]. Given a timed diagnostic trace

$$\begin{aligned}\tau = \ell_0 &\xrightarrow{d_0, e_0} \ell_1 \cdots \\ &\xrightarrow{d_{n-1}, e_{n-1}} \ell_n\end{aligned} \tag{4}$$

a trace path is encoded as a conjunction of linear arithmetic constraints:

$$\begin{aligned}\mathrm{Path}(\tau, \theta, \mathbf{z}_\tau) = &\, \mathit{Init}(\nu_0) \\ \wedge \bigwedge_{i=0}^{n-1} &\Big(d_i \geq 0 \wedge \mathit{Inv}(\ell_i, \nu_i + d_i, \theta) \\ &\wedge \mathit{Guard}(e_i, \nu_i + d_i, \theta) \wedge \mathit{Reset}_i \Big)\end{aligned} \tag{5}$$

Here, $\mathit{Init}(\nu_0)$ initializes the clock valuations, while d_i, Inv, Guard, and Reset_i define the non-negative delays, location invariants, transition guards, and clock resets at step i, respectively, under the modified bounds θ. The original violation encoding can then be viewed as the conjunction of path executability and the final property violation:

$$\mathrm{Enc}(\tau, \varphi, \theta) = \mathrm{Path}(\tau, \theta, \mathbf{z}_\tau) \wedge \mathit{Bad}_\varphi(\ell_n, \nu_n) \tag{6}$$

The predicate $\mathit{Bad}_\varphi(\ell_n, \nu_n)$ characterizes the violation state of φ at the final location. To block this violating behavior while preserving executability of the discrete path skeleton, the single-trace repair obligation uses two copies of the trace variables:

$$\begin{aligned}\Psi^{\mathrm{exec}}_{\tau,\varphi}(\theta) = &\, \mathrm{QE}\Big(\exists \mathbf{z}^+_\tau .\, \mathrm{Path}(\tau, \theta, \mathbf{z}^+_\tau) \\ &\wedge \neg \exists \mathbf{z}^-_\tau . \Big(\mathrm{Path}(\tau, \theta, \mathbf{z}^-_\tau) \\ &\wedge \mathit{Bad}_\varphi(\ell^-_n, \nu^-_n) \Big) \Big)\end{aligned} \tag{7}$$

The first part requires that the original discrete path skeleton still has at least one timed realization after repair. The second part requires that no assignment of delays and clock valuations can make that same path violate φ.

We extend this single-trace formulation to a joint, multi-property incremental setting. At refinement round k, counterexamples from all currently violated properties are accumulated into a trace pool

$$\mathcal{T}_k = \{(\tau_j, \varphi_j)\}_{j=1}^{m_k} \tag{8}$$

MPTA-Repair assembles a joint MaxSMT problem with the constraint system defined as

$$\begin{aligned}\Phi_k(\theta) = &\mathit{Dom}(\theta) \wedge \mathit{Block}_k(\theta) \\ &\wedge \bigwedge_{(\tau,\varphi) \in \mathit{Select}(\mathcal{T}_k)} \Psi^{\mathrm{exec}}_{\tau,\varphi}(\theta)\end{aligned} \tag{9}$$

Here, $\mathit{Dom}(\theta)$ enforces static interval consistency and domain limits, such as $b'_s \geq 0$, and $\mathit{Select}(\mathcal{T}_k)$ represents a reduced subset of traces obtained after relation-aware filtering. When an instantiated candidate model fails verification, the constraint

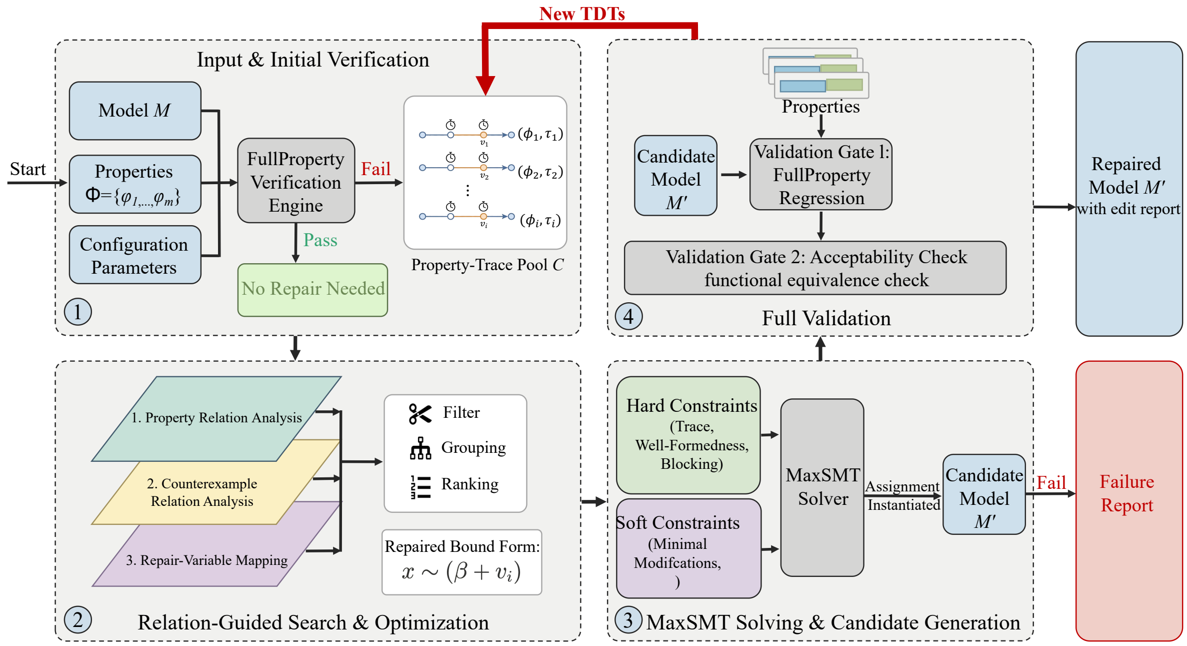


Fig. 2. Overview of the MPTA-Repair workflow.

system is updated incrementally. The framework expands the pool via

$$\mathcal{T}_{k+1} = \mathcal{T}_k \cup NewTDT(M(\theta_k), \Phi) \tag{10}$$

and updates the search-space block to exclude the failed assignment θ_k over the set of repairable bounds S:

$$Block_{k+1}(\theta) = Block_k(\theta) \wedge \left(\bigvee_{s \in S} \delta_s \neq \delta_s^k \right) \tag{11}$$

where δ_s^k is the valuation assigned to variation variable δ_s in round k.

The optimization objective is governed by a hierarchical set of soft constraints. MPTA-Repair uses a lexicographic objective that first favors small repairs and then incorporates risk- and coverage-based priorities:

$$\operatorname{lex\,min}_{\theta} \begin{pmatrix} \sum_{s \in S} \mathbf{1}\{\delta_s \neq 0\}, \\ \sum_{s \in S} |\delta_s|, \\ \sum_{s \in S} risk_s \cdot \mathbf{1}\{\delta_s \neq 0\}, \\ -\sum_{s \in S} benefit_s \cdot \mathbf{1}\{\delta_s \neq 0\} \end{pmatrix} \tag{12}$$

Here, $\mathbf{1}\{\cdot\}$ denotes the indicator function, yielding 1 when the inner condition holds and 0 otherwise. The primary and secondary objectives minimize the number and total magnitude of altered clock bounds to produce concise repairs. The third term incorporates a penalty coefficient $risk_s$ to discourage modifications prone to regression, while the final term uses a reward coefficient $benefit_s$ to prioritize repair sites that cover a broader set of violated traces.

C. Relation-Guided Search Optimization

During successive refinement rounds, the accumulation of violated properties, diagnostic traces, and editable clock bounds significantly expands the search space [17]. Unstructured encoding of all property-trace constraints and repairable bounds increases the complexity of the MaxSMT problem and can lead to repeatedly solving similar constraints. To mitigate this overhead, MPTA-Repair introduces relation-guided optimization to filter, group, and rank repairable variables and constraints. Crucially, these relation-based heuristics serve strictly as search optimizations; validity and admissibility of generated candidates are still guaranteed through full-property regression verification and admissibility checking.

At the property layer, MPTA-Repair operates on timed safety properties normalized into an `A[]` fragment over location predicates and clock or integer constraints [7]. Relational analysis employs conservative syntactic and structural rules to evaluate equivalence and dominance. Two properties are defined as equivalent if and only if their normalized representations are syntactically identical. Dominance is established only when clear logical subsumption holds, for example when two upper-bound properties share identical structures but differ in their numerical thresholds and the tighter bound implies the weaker one. For instance, `A[]` $x \leq 5$ subsumes `A[]` $x \leq 10$ under the same structural context. Consequently, MPTA-Repair can retain only the stronger properties for the subsequent solving phase to optimize efficiency.

At the counterexample layer, the framework maintains a managed pool of timed diagnostic traces (TDTs) indexed by their corresponding violated properties. Redundant traces are eliminated via exact-duplicate removal. The remaining TDTs are annotated with structural features, including traversed transitions, visited locations, active clocks, guard or invariant bounds, and violated properties from which they are generated [11]. Traces sharing identical discrete transition sequences or overlapping clock zones are grouped together to expose the common structural bottlenecks of the model. A trace constraint is omitted from the active encoding only when it is an exact duplicate or when a sound subsumption check shows that its local constraint pattern is already covered by an encoded trace constraint.

At the repair-bound layer, MPTA-Repair establishes a mapping between each active property-trace constraint and the clock bounds that occur in or affect the encoded trace constraints. This mapping restricts the set of active decision variables in the MaxSMT problem to those directly relevant to the current counterexample pool, while providing a heuristic ordering for candidate generation. Rather than imposing rigid structural constraints or hard-coding specific modifications, these relational scores serve as an adaptive guidance mechanism. They improve repair efficiency by ranking the repair variables, prioritizing those whose modifications are more likely to satisfy the joint constraint system.

D. Validation, Admissibility, and Refinement

In MPTA-Repair, candidate validation determines whether a generated edit set can be accepted as a repair [18]. After a candidate edit set is generated, the repaired model is verified against the entire property set Φ. If a candidate resolves the observed violation but induces new violations against the specification, it is rejected, indicating that the current property-trace pool is insufficient. MPTA-Repair then extracts the newly observed diagnostic trace and appends it to the pool, guiding the subsequent refinement round.

Candidates that pass full-property regression verification are further evaluated against an admissibility check to maintain functional equivalence [11]. This step ensures that clock-bound modifications do not satisfy properties by disabling original functional behaviors. Rather than enforcing timed-language equivalence, which would conflict with the goal of eliminating violating timed paths, admissibility is verified by constructing untimed automata from both the original and repaired models and comparing their accepted untimed languages [19]. A candidate is rejected if it adds or removes discrete functional paths, ensuring that only timing characteristics are modified.

The refinement loop therefore combines local candidate generation with global validation. Local MaxSMT formulations propose candidate assignments, full-property regression ensures complete specification compliance, and functional-equivalence checking maintains the original functional equivalence.

IV. Experimental Evaluation

This section evaluates MPTA-Repair on both benchmark and industrial case studies. The evaluation follows the clock-bound repair setting defined in Section III, where only numerical bounds in guards and invariants are repairable. For comparison, we evaluate MPTA-Repair against an iterative baseline extended from the TARTAR paradigm [11].

TABLE I
Statistics of the benchmark and industrial datasets.

Dataset	Original models	Mutated models	Avg. properties/model
Benchmark	9	54	4.1
Industrial	5	105	7.3

TABLE II
Repair results across benchmark and industrial datasets.

Dataset	Method	Mutated models	Repaired	Success rate	Avg. time
Benchmark	Iterative baseline	54	39	72%	60.61s
	MPTA-Repair	54	50	93%	15.46s
Industrial	Iterative baseline	105	21	20%	35.59s
	MPTA-Repair	105	80	76%	85.28s

A. Evaluation Strategy and Datasets

We evaluate MPTA-Repair on two datasets, summarized in Table I. The benchmark dataset includes nine standard models from UPPAAL [20] and the XtaBenchmarkSuite [21], such as Elevator and FDDI. The industrial dataset contains five closely coupled case studies: Gear Control System for automotive transmission [22], SAE RoR for autonomous-driving decisions [23], Train Controller Alarm for railway emergency timing [24], Train Gate Controller for shared-resource concurrency [1], and Pacemaker for medical pacing [25]. Compared with the benchmark models, these case studies involve more tightly coupled timing constraints across multiple properties.

Since real-world timed-automata models with labeled clock-bound faults are difficult to obtain, we synthesize faulty instances using mutation [26], retaining only mutants that are syntactically valid, violate at least one timing safety property, and preserve the untimed behavior of the original model. Each repair instance is executed with a 10-minute timeout for both MPTA-Repair and the baseline.

We compare MPTA-Repair against an adapted iterative TARTAR-style baseline [11], which was originally designed for single-property repair and operates via trace-local repair generation. This comparison allows us to examine whether using a joint counterexample pool improves repair robustness over localized trace analysis.

B. Experimental Results

The experimental evaluation demonstrates that MPTA-Repair outperforms the iterative baseline, particularly on complex industrial models. As shown in Table II, the baseline achieves a 72% success rate on the benchmark dataset, whereas MPTA-Repair reaches 93%. This performance gap widens on the industrial dataset, where the baseline repairs only 20% of the cases compared to 76% for MPTA-Repair. The baseline reports a lower average duration on the industrial dataset because unsuccessful runs typically terminate early due to timeouts.

The baseline remains competitive on simpler benchmarks, where injected timing faults are often localized to a small number of guards or invariants. However, its performance drops significantly on the industrial dataset, where concurrent component interactions and tightly coupled timing constraints cause trace-local repairs to fail global validation by producing candidates that violate admissibility or regression requirements.

MPTA-Repair achieves higher success rates by integrating constraints from multiple counterexamples and properties into a joint generation process. Consequently, the framework produces coordinated, small-scale modifications that successfully satisfy both full-property regression and admissibility validations.

C. Discussion and Practical Lessons

The evaluation highlights two primary lessons for applying automated repair to timed automata. First, full-property regression verification is necessary because industrial properties are often interdependent; a localized fix for one response-time property can inadvertently violate another safety property, such as maximum waiting bounds. MPTA-Repair avoids these regressions by treating property restoration as a global objective. Second, admissibility is critical to distinguish valid repairs from behavior-breaking edits that satisfy timing properties by disabling intended functional behaviors, such as preventing critical system actions. The admissibility check ensures that property satisfaction is not achieved at the expense of functional behavior.

Nevertheless, the framework can still fail to generate repairs in certain cases when it fails to find an admissible candidate within the time budget. This limitation is prominent in complex models like the Pacemaker case study, where tightly coupled timing constraints expand the search space, causing the solver to time out before finding a valid candidate.

V. Conclusion

This paper presented MPTA-Repair, an automated workflow for multi-property and multi-counterexample clock-bound repair of industrial timed automata. To overcome the limitations of isolated single-trace fixes, the method accumulates diagnostic traces and exploits explicit relations among properties, counterexamples, and repairable bounds to guide a MaxSMT-based search. Each synthesized candidate is validated through full-property regression and an admissibility check based on untimed functional equivalence [11]. This design ensures that accepted repairs are system-level timing corrections that preserve the intended operational behavior.

Evaluations across benchmark and industrial datasets show that MPTA-Repair outperforms the iterative baseline, particularly in industrial systems with dense property interactions. The failure analysis further indicates that unsuccessful repairs are mainly caused by the difficulty of finding an admissible candidate within the given time budget, especially when dense timing interactions substantially enlarge the clock-bound repair search space.

Future work will primarily focus on extending the automated repair space beyond numerical clock bounds to include broader structural modifications, such as altering synchronization labels and redesigning discrete control logic. We also plan to investigate the theoretical and algorithmic relationship between candidate repair generation and admissibility verification. By formally integrating behavioral admissibility constraints into the early stages of search-space formulation, we aim to proactively prune functionally invalid candidates and thereby improve computational efficiency for large-scale industrial applications.

References

[1] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[2] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[3] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer: Applying formal methods to railway standard interfaces," in *Proceedings of Formal Methods for Industrial Critical Systems*, ser. Lecture Notes in Computer Science, vol. 12863. Springer, 2021, pp. 174–190.

[4] H. Lönn and P. Pettersson, "Formal verification of a TDMA protocol start-up mechanism," in *Proceedings of the 1997 IEEE Pacific Rim International Symposium on Fault-Tolerant Systems*, 1997, pp. 235–242.

[5] Z. Jiang, M. Pajic, S. Moarref, R. Alur, and R. Mangharam, "Modeling and verification of a dual chamber implantable pacemaker," in *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, ser. Lecture Notes in Computer Science, vol. 7214. Springer, 2012, pp. 188–203.

[6] M. Mikučionis, K. G. Larsen, J. I. Rasmussen, B. Nielsen, A. Skou, S. U. Palm, J. S. Pedersen, and P. Hougaard, "Schedulability analysis using Uppaal: Herschel-planck case study," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, ser. Lecture Notes in Computer Science, vol. 6416. Springer, 2010, pp. 175–190.

[7] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[8] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[9] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[10] A. E. Dalsgaard, R. R. Hansen, K. Y. Jørgensen, K. G. Larsen, M. C. Olesen, P. Olsen, and J. Srba, "opaal: A lattice model checker," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 487–493.

[11] M. Kölbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," in *International Conference on Computer Aided Verification*. Springer, 2019, pp. 79–96.

[12] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*. ACM, 1993, pp. 592–601.

[13] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, ser. Lecture Notes in Computer Science, vol. 11823. Springer, 2019, pp. 129–146.

[14] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," *Logical Methods in Computer Science*, vol. 18, no. 3, pp. 12:1–12:32, 2022.

[15] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 6806. Springer, 2011, pp. 504–509.

[16] R. Sebastiani and S. Tomasi, "Optimization modulo theories with linear rational costs," *ACM Transactions on Computational Logic*, vol. 16, no. 2, pp. 12:1–12:43, 2015.

[17] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proceedings of Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 1855. Springer, 2000, pp. 154–169.

[18] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[19] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.

[20] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[21] R. Farkas and G. Bergmann, "Towards reliable benchmarks of timed automata," in *Proceedings of the 25th PhD Mini-Symposium*, 2018, pp. 20–23.

[22] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 1384. Springer, 1998, pp. 281–297.

[23] G. V. Alves, L. A. Dennis, and M. Fisher, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Journal of Sensor and Actuator Networks*, vol. 10, no. 3, p. 41, 2021.

[24] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," *SIMULATION: Transactions of The Society for Modeling and Simulation International*, vol. 101, no. 10, pp. 1057–1076, 2025.

[25] Z. Jiang, M. Pajic, A. Connolly, S. Dixit, and R. Mangharam, "Real-time heart model for implantable cardiac device validation and verification," in *Proceedings of the Euromicro Conference on Real-Time Systems*. IEEE Computer Society, 2010, pp. 239–248.

[26] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.